Deriving Classic Recommendation Models and Losses, Part 2

Historical States, Interest Specialization, and Ranking Gradients

Bo Hou

2026-10-30

This article continues Part 1 with user-history encoding, multi-interest retrieval, MoE-inspired routing balance, and pointwise, pairwise, and listwise objectives. It preserves the substantive questions, objections, and corrections from the discussion instead of reducing the discussion to a catalogue of model names.

Dialogue passages are edited reconstructions rather than verbatim transcripts. Original-paper definitions, simplified teaching models, and extensions proposed during the discussion are distinguished explicitly. This is a technical learning notebook, not a claim of research novelty or of experimentally validated industrial performance.

Previous article: Part 1, English PDF; Part 1, Chinese PDF.

Contents

1. Where Part 1 left off: four separate questions
2. The starting objection: we already have self-attention
3. Designing a GRU: what to write versus how much to write
4. Information loss and the gap between mechanism and intended meaning
5. DIEN: give intermediate states an explicit supervision task
6. Window construction and the main prediction target
7. SASRec: align a familiar causal encoder with its original objective
8. Multiple interests: replace one user vector with K vectors
9. Hard selection, soft assignment, and collapse
10. Representation diversity and routing balance are different problems
11. Borrowing from MoE: sparse per input, useful coverage overall
12. Switch balancing: why multiply two different proportions?
13. What the original MIND paper actually does
14. Does massive data solve unused interests automatically?
15. A compact map and timeline of user models
16. What common benchmarks establish, and what they do not
17. Loss categories describe supervision, not scalar output shape
18. Pairwise logistic: from a training triple to both score gradients
19. Hinge, BPR, and metric-weighted ranking gradients
20. Listwise objectives are not all simulated NDCG gradients
21. Separate architecture, data, objective, and optimization in experiments
22. Interview preparation and a practical ML coding loop
23. Correction log, unfinished details, and references

1. Where Part 1 left off: four separate questions

Part 1 covered two-tower retrieval, history representations, FM, DeepFM, Wide & Deep, DCN v1/v2, DLRM, and DIN. Before continuing, I revised the framework used to understand a learning system.

Originally it was "data, model, optimization." The loss deserves its own category:

Component	What it determines	What cannot replace it
Data	Examples, available information, supervision, observation distribution	A new architecture cannot create missing observations
Architecture	Representable functions and the relationships made easy to express	Expressive power does not specify the desired behavior
Loss	Which outcomes are rewarded and which errors are penalized	An optimizer does not independently know the business objective
Optimization	Parameter updates, stability, and computational practicality	A sensible objective is not necessarily easy to optimize

Evaluation and deployment close the loop: is the improvement reliable and useful, and can the system compute, refresh, and monitor it online?

My conclusion: The same scoring network can use BCE or a pairwise objective. Its architecture may remain unchanged while its learning target changes. Even the optimizer can remain identical. The objective is therefore not merely an implementation detail under optimization.

2. The starting objection: we already have self-attention

The discussion resumed from DIN. Without order or time information, a candidate-conditioned weighted sum does not change merely because the order of otherwise identical historical events changes.

Consider:

- User A watched comedies earlier and several thrillers recently.
- User B watched thrillers earlier and several comedies recently.

Their historical sets can be identical even when their current interests differ.

Prompt: How would you make each historical representation contain what happened before it?

My objection: Isn't this what self-attention already does? The last valid token can contain information from the preceding history.

That objection was correct. We had already derived a causal history encoder in Part 1. There was no need to pretend that sequence modeling remained unsolved merely to introduce another paper name.

The next useful questions were more specific:

1. GRU is an alternative history encoder. What does its recurrence actually mean?
2. Whether the encoder uses GRU or attention, how can intermediate states receive more direct supervision?

A causal mask controls which positions are visible. Positional or temporal inputs carry order information. Saying "it has attention" does not by itself establish that the input order is represented.

3. Designing a GRU: what to write versus how much to write

3.1 Start with the smallest recurrence

$$h_t = \mathrm{GRU}(x_t, h_{t-1}).$$

Here x_t is the current behavior representation, h_{t-1} summarizes earlier history, and h_t is the updated state after reading the current event.

My interpretation: Feed an input through a network, obtain an output, combine the previous information with the next input, and repeat.

Correction: The recurrence carries the previous hidden state, not simply the previous raw input. The state is the learned summary. The GRU parameters are shared across time; the state and current input change.

This is not a separate trained network at every time step. Nor is the memory the optimizer's newly updated parameter tensor. Recurrent state and model parameters are different objects.

3.2 My proposed gate design

I suggested concatenating $[h_{t-1}, x_t]$, using a gate network to decide which old information to retain, generating new content through another transformation, and combining the two into h_t.

Fix the notation used in this article:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z),$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t.$$

Here z is the **write proportion for candidate content**, and 1-z is the retention proportion. Some texts and frameworks reverse that naming convention. Read the full recurrence rather than assuming that a variable called an update gate always weights the same term.

Typically z has one value per state coordinate. It need not be a single scalar applied to the entire state.

3.3 "Is sigma ReLU, activating above a threshold?"

In this equation sigma specifically means sigmoid:

$$\sigma(a) = \frac{1}{1 + e^{-a}} \in (0, 1).$$

It is a smooth proportion, not a hard threshold:

- z=0.9 favors candidate content in that coordinate.
- z=0.1 favors retaining the old coordinate.

ReLU is unbounded and cannot directly be interpreted as this convex-mixture proportion. Sigmoid also saturates near its extremes; adding gates does not eliminate every gradient problem.

3.4 Where does the candidate state come from?

A first teaching construction is

$$\tilde{h}_t = \tanh(W_h[h_{t-1}, x_t] + b_h).$$

The responsibilities are distinct:

- $\tilde{h}_t$: what content could be written?
- z_t: how much should be written?

Tanh is the hyperbolic tangent, bounding candidate coordinates in (-1,1). Its name does not contain an extra learnable H parameter.

My guess: Does a coordinate near plus or minus one mean that the information is particularly important?

Correction: Activation magnitude is not a definition of semantic importance. A coordinate's task value depends on what downstream computation does with it. It is not valid to read "important interest" directly from a large absolute activation.

3.5 Introduce the reset gate only after the need is clear

When generating new content, perhaps not all of the old state should participate. Introduce

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r),$$

$$\tilde{h}_t = \tanh\left(W_h[r_t \odot h_{t-1}, x_t] + b_h\right).$$

This is a common GRU formulation. Implementations can differ in where reset multiplication occurs relative to linear transformations and biases.

The reset gate controls how old state participates in **constructing candidate content**. The update gate controls the **final mixture** of old and candidate content. Deriving the two responsibilities is more useful than memorizing a list of gate names.

4. Information loss and the gap between mechanism and intended meaning

4.1 The initial uncertainty

My question: GRU feels somewhat related to attention, but doesn't a fixed state lose information? Isn't that also the recurring concern with RNNs and LSTMs?

Separate two difficulties:

1. **Representation compression:** more and more history is written into a finite-dimensional state, potentially discarding detail.
2. **Optimization paths:** gradients travel through many recurrent transitions and can shrink or grow.

Gates and residual-like updates help create useful information paths. They do not guarantee retention of everything relevant.

Attention can retain representations at separate positions and access relevant positions more directly. The fixed-length encoder bottleneck was one motivation for early encoder-decoder attention.

But attention also has finite context, finite dimension, aggregation, and objective-dependent learning. It is not a lossless memory device.

4.2 An explicit computation does not guarantee an intended interpretation

I felt that FM/DCN were easier to connect to their modeling hypotheses: they explicitly calculate inner products or coordinate products. Calling a GRU state an "interest state" felt less convincing.

That distinction is useful:

- FM explicitly computes factorized interactions; DCN explicitly constructs products.
- GRU explicitly updates a state through gates.
- Self-attention explicitly computes learned matching and weighted aggregation.

Those are facts about the computation. Saying that a coordinate is comedy, a gate is long-term preference, or QK similarity is necessarily semantic requires evidence.

A product is not itself a statistical correlation coefficient. An attention score is not automatically the semantic relation a human would choose.

4.3 Why intermediate supervision is a natural next step

My inference: The recurrent cell is small, but unrolling creates a deep computation. Perhaps a cost on intermediate steps can encourage the state to retain the information we actually want.

Yes, with a qualification. An auxiliary objective usually supervises a **prediction made from a state**, not a human interpretation of the gate itself.

Make "this state represents current interest" operational: ask it to distinguish an actual next interaction from an alternative. That supplies training pressure. It does not prove that the internal coordinates acquire a particular interpretation.

5. DIEN: give intermediate states an explicit supervision task

5.1 Rephrase the real question

If there is only one final click label, intermediate states receive much of their signal indirectly. Can each state h_t also predict something observable?

Construct

$$(h_t, v_{t+1}^+, v_{t+1}^-),$$

where the positive is the actual next clicked item and the negative follows a stated sampling protocol.

My immediate answer: Of course: make the state match the next actual click and differ from the sampled negative. Use inner product or cosine, then softmax cross-entropy.

That is a valid design. It must be distinguished from the exact objective in the original DIEN model.

5.2 Align a teaching objective with the paper

Define a shared binary head for illustration:

$$g_\psi(h_t, v) = \mathrm{MLP}_\psi([h_t, v]), \qquad D_\psi(h_t, v) = \sigma(g_\psi(h_t, v)).$$

Then

$$\ell_t^{\text{aux}} = -\log D_\psi(h_t, v_{t+1}^+) - \log\left(1 - D_\psi(h_t, v_{t+1}^-)\right).$$

This is next-behavior discrimination: the state should contain information that helps distinguish a real continuation from a negative. BCE with an MLP head does not necessarily imply a smaller Euclidean distance. "Closer" should then mean better task compatibility.

The original DIEN uses positive/negative next-behavior supervision at intermediate steps together with the main CTR loss. Its public-data and industrial negative sources differ; they should not all be described as randomly selected unclicked items.

5.3 Masks and reduction define the weighting

Let m_{bt} indicate whether a time position is valid. One possible teaching objective averages over valid transitions:

$$L_{\text{aux}} = \frac{\sum_{b,t} m_{bt} \ell_{bt}^{\text{aux}}}{\sum_{b,t} m_{bt}}, \qquad L = L_{\text{CTR}} + \lambda_{\text{aux}} L_{\text{aux}}.$$

Another reduction averages within each user and then across users. That weights long and short histories differently. The difference should not be dismissed as a meaningless implementation detail.

Padding is not an observed negative. Positions without a valid next event should not receive invented supervision.

5.4 Full DIEN has another component

Our "GRU plus auxiliary loss" discussion aligns with the interest extractor, not the whole original architecture. DIEN also contains target-conditioned interest evolution. Its AUGRU uses attention strength to modulate a recurrent update gate.

Thus a GRU hidden state feeding a two-tower retriever is a sensible model, but not automatically original DIEN. Its candidate-conditioned evolution component naturally belongs in ranking rather than a candidate-independent ANN query encoder.

6. Window construction and the main prediction target

I connected this back to the data pipeline: take 100 user events, make windows of length 30, supervise internal next-step transitions, and use the final event as a main target.

First define whether the target is counted inside the window length.

6.1 Protocol A: the thirtieth event is the target

$$[a_1, \ldots, a_{29}] \longrightarrow a_{30}.$$

- Encode the first 29 events into $h_1, \ldots, h_{29}$.
- Internal auxiliary tasks may use h_t to predict a_{t+1}, for example t=1 through 28.
- The thirtieth interaction can define a positive main candidate with sampled main negatives, or the main task can use actual impression click labels.

One can also apply auxiliary supervision from h_{29} to a_{30}, but that is additional, potentially overlapping supervision and needs explicit weighting. Event 30 must not enter the history encoder before the model predicts it.

6.2 Protocol B: all thirty events are history

$$[a_1, \dots, a_{30}] \longrightarrow a_{31}.$$

From 100 events, this creates 70 fixed-history next-event examples. Enumerating all complete 30-event segments under Protocol A creates 71 segments. The difference is endpoint counting, not an architectural disagreement.

Training examples can be shuffled. Forward-time evaluation still needs global partition boundaries: a later training window must not silently contain an earlier test target. User states, statistics, and pretrained encoders must also satisfy the chosen information-time protocol.

7. SASRec: align a familiar causal encoder with its original objective

We had already derived the main construction: item embeddings and positions enter a causal self-attention encoder; each prefix state h_t predicts a next item.

A simplified score is

$$s_{tj} = h_t^\top e_j.$$

Training can use separately constructed prefixes or supervise multiple valid positions in a sequence forward pass. Serving often uses the final valid state as the current user representation.

An important alignment detail: **original SASRec uses binary cross-entropy with sampled negatives**. Being a Transformer does not automatically imply full-item softmax training. Later implementations can change the loss or sampler, and that can change the results.

Keep four things separate:

- Architecture: causal self-attention history encoder.
- Prediction task: next-item prediction.
- Original objective: positive versus sampled-negative BCE.
- Extensions: other CE/softmax and negative-sampling protocols.

Similar architectures do not establish identical training objectives.

8. Multiple interests: replace one user vector with K vectors

8.1 The natural output change

Given user input X_u, generate

$$H_u = f_\theta(X_u) \in \mathbb{R}^{K \times d}.$$

This is **K vectors of dimension d**, not one K-dimensional vector.

The output can come from a history encoder with several heads or another network. These vectors are intermediate activations, and backpropagation updates the network that produces them.

8.2 “Isn’t it just a lookup table?”

My initial picture was to generate K alternatives, choose the maximum-matching index, and let CE update that entry.

Two different parameterizations must be separated:

1. A user ID retrieves a directly learned K-by-d table entry; gradients update stored parameters.
2. History and profile enter an encoder that outputs K-by-d activations; gradients travel through those activations into the upstream network.

The subsequent discussion concerns the second design. It is inconsistent to call the vectors dynamically generated and then explain their learning as if each were an independently stored per-user parameter.

8.3 "It feels like K-means centers"

That is a useful intuition: behaviors are points, interests resemble centers, behaviors receive assignments, and their representations are aggregated.

But distinguish:

- A network that outputs K vectors in one pass without explicit clustering.
- A network with repeated assignment-and-aggregation steps.
- The outer optimizer that updates parameters from recommendation loss.

Routing iterations inside a forward pass and optimizer steps across batches are two different meanings of iteration.

9. Hard selection, soft assignment, and collapse

9.1 Which interest should match a positive?

Question: When a user clicks a video, should every interest vector approach it?

My answer: Let the most relevant one approach, and move all the others away.

Correction: The first part is sensible; the second is too strong. A non-winning interest is not automatically a negative. It may also be relevant, so its losing the competition does not justify repelling it.

A teaching baseline is

$$s(u, v) = \max_k h_{u,k}^\top e_v.$$

Use that score in positive/negative candidate training. For a given user-item pair, hard max sends its direct selection gradient mainly through the winner. Other items can select different interests.

9.2 What does collapse mean?

Failure	Observable behavior	Consequence
Usage imbalance	Almost every input selects the same branch	Other branches receive little direct task signal
Representation redundancy	K vectors have similar directions or score patterns	Rotating winners still fails to create distinct interests

Similar distances to one particular video do not prove collapse. More convincing evidence is near-identical behavior over many candidates or persistent lack of participation by most branches.

9.3 Initialization and reinforcement

I proposed random initialization to separate the starting points.

That breaks symmetry but does not guarantee lasting specialization. A branch with a small early advantage can win more often and receive more updates. Identical initialization can also start all branches in the same state.

Ordinary random initialization is not necessarily Bayesian initialization. Bayesian terminology needs an explicit prior, posterior, or corresponding inference procedure.

9.4 Temperature changes assignment weights

One candidate-dependent soft assignment is

$$a_k(u,v) = \frac{\exp(h_{u,k}^\top e_v/\tau)}{\sum_j \exp(h_{u,j}^\top e_v/\tau)}.$$

- Low temperature sharpens the assignment.
- High temperature spreads the assignment.

This is soft assignment or soft routing, not a soft label. The observed label has not thereby become probabilistic supervision.

One can aggregate $\bar{h} = \sum_k a_k h_k$ before scoring, or choose another smooth maximum formulation. Writing softmax alone does not define a complete algorithm: specify how its output enters the objective.

Soft aggregation creates gradient paths to more branches, but gradients may still be tiny, and the branches may become similar. It is not an anti-collapse guarantee.

10. Representation diversity and routing balance are different problems

10.1 "Push the centers farther apart"

I suggested regularization that increases distances between the K centers.

Without controlling magnitude, the network can enlarge norms instead of learning different directions. To compare directions, first normalize:

$$\hat{h}_k = \frac{h_k}{\|h_k\|_2 + \epsilon}.$$

This removes overall vector scale. It does not freeze representations, and it is not per-coordinate feature standardization.

An **optional teaching regularizer** is

$$L_{\text{div}} = \frac{1}{K(K-1)} \sum_{k \neq j} (\hat{h}_k^\top \hat{h}_j)^2.$$

It encourages approximately orthogonal directions. It is not the original MIND objective. If K exceeds d, pairwise orthogonality is impossible; related real interests also need not be forced orthogonal. Its form and strength require empirical justification.

10.2 "Normalization still doesn't update an unused branch"

Exactly. Normalization addresses scale. A diversity penalty addresses similar representations. Routing balance addresses participation. None is a substitute for all the others.

Perfectly balanced routing can select a collection of identical vectors. Distinct vectors can remain unused. A single health metric cannot capture every failure mode.

11. Borrowing from MoE: sparse per input, useful coverage overall

The analogy to MoE was natural: if a gate chooses experts, training must monitor whether only a few experts are ever selected.

Separate two levels:

- For one input, top-k activates only a few experts to control compute.
- Across a batch or longer interval, avoid having most experts idle while a few receive nearly everything.

The goal is not a maximally uniform distribution for each individual input. That would undermine the purpose of sparse expert computation.

In multitask settings, expert overlap between tasks can be measured. Less overlap is not inherently better: related tasks may benefit from sharing an expert.

Useful diagnostics include selection counts, average soft probability, gradient norms, persistent idle rates, output similarity, and overlap among retrieved items. With capacity limits, inspect executed load as well as pre-capacity routing.

Boundary of the analogy: MoE experts are globally shared computational modules. Multi-interest outputs are often generated per user, and slot identities need not have stable semantic meanings across users. Borrowing a balancing mechanism is an architectural choice, not an equivalence between the two systems.

12. Switch balancing: why multiply two different proportions?

12.1 "Can't we estimate it with MLE?"

For estimating categorical usage frequencies, yes:

$$f_k = \frac{n_k}{B}.$$

If 100 inputs dispatch 90, 8, and 2 examples to three experts, their observed proportions are 0.90, 0.08, and 0.02.

That measures the current distribution. It does not by itself supply a differentiable learning mechanism. Hard argmax counts are locally constant almost everywhere, so ordinary backpropagation needs a surrogate or a different assignment procedure.

12.2 Why f and average probability are not the same

First use top-1, avoiding ambiguity about top-2 normalization. Let $p_k(x)$ be the softmax probability before selecting an expert:

$$f_k = \frac{1}{B}\sum_x \mathbf{1}[\arg\max_j p_j(x) = k], \qquad P_k = \frac{1}{B}\sum_x p_k(x).$$

Suppose all 100 inputs assign A/B probabilities 0.51/0.49:

$$f = (1, 0), \qquad P = (0.51, 0.49).$$

Top-1 takes the maximum; it does not randomly draw A with probability 0.51. One quantity counts actual decisions. The other averages pre-decision soft probabilities.

For multiple routes per input, define the denominator of f and any gate renormalization explicitly. Do not copy a top-1 equation into top-2 without stating the convention.

12.3 The most intuitive uniformity penalty

A simple constructed regularizer is

$$L_{\text{simple}} = \sum_k (P_k - 1/K)^2.$$

It encourages uniform average probabilities. But near-uniform probabilities do not imply balanced hard choices: the 0.51/0.49 example already demonstrates this.

This does not require uniform probabilities for every input. Different examples can make sharp but different selections whose aggregate is balanced.

12.4 The Switch-style surrogate

Switch uses the auxiliary term

$$L_{\text{bal}} = \lambda K \sum_k f_k P_k.$$

Backpropagation goes through P while f is treated as a stopped-gradient observed routing statistic. It penalizes assigning large soft probability to experts that currently receive a large fraction of hard assignments.

My objection: Why not cosine or triplet? Why multiply the proportions directly?

If cosine merely encourages two distributions to agree, both can concentrate on the same expert and agree perfectly while remaining badly imbalanced. A triplet objective expresses relative distances among positive and negative objects; this problem does not naturally supply such a triple.

The dot product is more naturally read as a cost: high observed load gives an expert a higher price for additional router probability.

12.5 Expand the gradient instead of stopping at the intuition

For the average probability,

$$\frac{\partial L_{\text{bal}}}{\partial P_k} = \lambda K f_k.$$

My reasoning: A larger f produces a larger derivative; gradient descent subtracts it and reduces the probability.

That captures the intended direction, but router parameters produce logits before softmax. Probabilities cannot be updated independently. For input x and logit a_{xj}, the chain rule gives

$$\frac{\partial L_{\text{bal}}}{\partial a_{xj}} = \frac{\lambda K}{B} p_j(x) \left[f_j - \sum_k f_k p_k(x) \right].$$

An expert above that input's probability-weighted average load receives downward pressure; one below it receives upward pressure. This is more precise than claiming that every logit independently decreases in proportion to its expert's load.

12.6 A surrogate is not a hard guarantee

If f is uniform and held fixed during that update, $\sum_k f_k P_k$ is constant for every valid probability vector P. The auxiliary router gradient is then zero.

Therefore this should not be described as a strict distance between two freely optimized distributions, or as a guarantee of equal training for every expert.

Noisy routing, balancing penalties, capacity limits, and quota-based assignment are different tools. A capacity limit alone may simply drop overflow tokens rather than train idle experts. A soft penalty does not impose a hard per-expert participation constraint.

13. What the original MIND paper actually does

At this point I asked to check the original paper, rather than let our proposed combination become remembered as the published method.

Original-paper alignment: MIND generates interests through behavior-to-interest dynamic routing, initializes routing logits with Gaussian randomness to break symmetry, and uses label-aware attention during sampled-softmax training. It does not introduce the Switch-style balancing term discussed above. Its experiments favor label-aware selection approaching hard attention for convergence and effectiveness.

Two routing questions must be separated:

1. Historical behaviors to interest centers: constructing the interests.
2. Interest centers to the training target: choosing relevant supervision paths.

To understand the first, use the following **conceptual iteration**, omitting the paper's squash nonlinearity and complete network:

$$w_{ik} = \mathrm{softmax}_k(b_{ik}), \qquad c_k = \mathrm{Aggregate}_i(w_{ik} T e_i),$$

$$b_{ik} \leftarrow b_{ik} + c_k^\top T e_i.$$

The idea is to assign by compatibility, aggregate, and reinforce agreement with the resulting center. These are forward computations. The outer recommendation objective trains transformation T, embeddings, and downstream parameters; an internal routing-logit update is not a separate optimizer step.

At serving time, generate several candidate-independent user vectors, retrieve neighbors for each, and merge the candidates. Training-time label-aware selection may use the target item, but the user-interest generator must not ingest future target behavior.

Correction from the discussion: Softer routing and balancing were design options we explored, not claims about the original MIND recipe or universally superior alternatives. The random-initialization suggestion did have a direct counterpart in the paper.

14. Does massive data solve unused interests automatically?

My argument: With billions or tens of billions of sessions and only a hundred interests, surely all of them should eventually have a chance to update. Perhaps this is not a major practical issue.

There is a reasonable point here. More diverse data can increase branch participation. We should not assume every large multi-interest model necessarily collapses.

But scale alone does not guarantee coverage. If routing persistently favors one branch, additional data may reinforce that advantage. Initialization, parameter sharing, the data distribution, and routing dynamics determine the opportunities, not merely the ratio of batch size to K.

Interest slots can also have permutation symmetry. Slot one for user A need not have the same semantic topic as slot one for user B. Slot statistics can diagnose some optimization failures without establishing global semantic specialization.

The practical response is to monitor a baseline rather than demand uniformity by default. Real interests have unequal strengths, and some users need fewer effective interests. Artificially dividing one real interest merely to fill K slots may not improve retrieval.

15. A compact map and timeline of user models

Method	User representation	Main expressive issue addressed
User ID lookup	A learned user vector	Remember long-term preference; limited direct response to new behavior
Historical pooling	Average or action/time-weighted summary	Use behavior evidence rather than only identity
GRU/LSTM encoder	Recurrent states	Order and selective memory
Self-attention / SASRec	Prefix-context states	Direct access to different historical positions
DIN	Candidate-conditioned aggregation	Different candidates emphasize different historical evidence
DIEN	Interest sequence and candidate-conditioned evolution	Intermediate supervision and interest change
Multi-interest / MIND	K user-interest vectors	Reduce compression of distinct preferences into one vector

A timeline is not a performance ranking:

Model	Date convention	Authors / team
GRU	2014	Cho and collaborators
GRU4Rec	2015 preprint / 2016	Hidasi and collaborators
DIN	2017 preprint / KDD 2018	Alibaba
SASRec	ICDM 2018	Kang and McAuley
DIEN	2018 preprint / AAAI 2019	Alibaba
MIND	CIKM 2019	Alibaba and collaborators

SASRec and DIEN were contemporary directions. Attention-based recommendation was not invented only after DIEN failed. The methods can be combined, and one does not automatically replace another.

The other line from Part 1 is FM (2010), Wide & Deep (2016), DeepFM/DCN (2017), DLRM (2019), and DCNv2 (2020 preprint / WWW 2021). It primarily concerns feature interactions rather than exactly the same user-history problem.

16. What common benchmarks establish, and what they do not

I wanted shared-data comparisons rather than a sequence of papers each reporting that its own architecture wins.

Tenrec provides one useful starting point. The following are author-published results under particular subset protocols, not a universal industrial ranking.

CTR-1M, shared embedding	AUC
NFM	0.7924
DCNv2	0.7922
DeepFM	0.7920
DIEN	0.7918
DCN	0.7911
DIN	0.7910
Wide & Deep	0.7910

Session recommendation, 1M	NDCG@20
SASRec	0.0194
GRU4Rec	0.0192
BERT4Rec	0.0185

Do not compare the two tables numerically across tasks. Point estimates alone also do not establish statistical significance. The shared-embedding protocol should not be mixed with the separate-embedding leaderboard.

In the original MIND Tmall experiment, HitRate@100 is 0.1648 for YouTube DNN, 0.1930 for one-interest MIND, and 0.2699 for five-interest MIND. This supports a multi-interest benefit in that setup, not a guaranteed gain on every dataset.

A 2024 reproducibility study further reports that SASRec does not beat GRU4Rec in every configuration. Model scale and evaluation protocol affect the comparison. Architecture, objective, sampling, and tuning need separate treatment.

17. Loss categories describe supervision, not scalar output shape

17.1 My common backpropagation interface

My observation: Each document eventually has a score. Once the loss supplies a derivative for each score, the same scoring function can backpropagate it.

Correct. With $s_i = f_\theta(q, d_i)$,

$$\frac{\partial L}{\partial \theta} = \sum_i \frac{\partial L}{\partial s_i} \frac{\partial s_i}{\partial \theta}.$$

But a scalar score per document does not make the objective pointwise. The classification depends on whether the loss couples an individual example, a relative pair, or an entire list.

For clarity, this article assumes a shared per-candidate scorer. A scorer using list context has additional structural dependencies between scores, but the full graph still follows the chain rule.

17.2 Three supervision organizations

Family	Example organization	What it emphasizes
Pointwise	Query-document with an individual label	Individual click probability or relevance
Pairwise	Two documents under the same query and a preference	Which result should precede the other
Listwise	A query and its candidate group	A joint relative distribution or permutation

BCE predicts the click process represented by the observation protocol. Position effects, negative sampling, and delayed labels can change that interpretation. A sigmoid alone does not create an unbiased preference probability.

18. Pairwise logistic: from a training triple to both score gradients

18.1 What exactly is in the batch?

My question: Do we calculate differences for every pair in a batch? Is one batch item a query, a result, or a pair?

The simplest record is

$$(q, d^+, d^-).$$

Under the same query, the preferred or more relevant result should score above the less preferred result. A batch can contain B such triples. Alternatively, it can contain several query lists and construct valid pairs within each list.

There is no requirement to enumerate every pair. Unrelated queries generally should not be paired directly. A document may participate in several pairs, which affects weighting and dependence.

18.2 Derive CE from a preference probability

$$s^+ = f_\theta(q, d^+), \quad s^- = f_\theta(q, d^-), \quad \Delta = s^+ - s^-.$$

Define the probability that the preferred item wins as

$$p_{+>-} = \sigma(\Delta).$$

For preference label one,

$$\ell = -\log \sigma(\Delta) = \log(1 + e^{-\Delta}).$$

This is the familiar pairwise logistic objective. A stable implementation uses softplus$(-\Delta)$.

For a soft preference target $y \in [0, 1]$, use BCE on the difference logit. Observed clicks do not always justify treating every preference as certain.

18.3 "Then send it back to the two scoring branches"

Exactly:

$$\frac{\partial \ell}{\partial \Delta} = \sigma(\Delta) - 1,$$

$$\frac{\partial \ell}{\partial s^+} = \sigma(\Delta) - 1, \qquad \frac{\partial \ell}{\partial s^-} = 1 - \sigma(\Delta).$$

Gradient descent tends to raise the positive score and lower the negative score. If delta is zero, the derivatives are -0.5 and +0.5. If the preferred item already wins comfortably, the pressure shrinks; a reversed order receives larger corrective pressure.

Both scoring paths usually share parameters:

$$\nabla_\theta \ell = (\sigma(\Delta) - 1)\left(\nabla_\theta s^+ - \nabla_\theta s^-\right).$$

Their contributions accumulate into one scorer. They are not necessarily independent pointwise models, and the gradients need not be implemented by hand.

18.4 A useful invariance

Add the same constant c to every score under a query. The pair difference remains unchanged:

$$(s^+ + c) - (s^- + c) = \Delta.$$

Pairwise logistic therefore does not require accurate absolute click probabilities. Ranking and calibration are distinct goals. They can be combined deliberately, but the objectives and possible tradeoffs should be acknowledged.

19. Hinge, BPR, and metric-weighted ranking gradients

19.1 Hinge / margin ranking

$$\ell_{\text{hinge}} = \max(0, m - (s^+ - s^-)).$$

Once the margin m is reached, that pair contributes zero gradient. Logistic decays smoothly and does not stop exactly at a finite positive difference.

Hinge is not L1 parameter regularization. One penalizes an insufficient ranking margin; the other typically penalizes parameter magnitude.

19.2 Similar formulas do not make RankNet and BPR identical methods

RankNet's preference probability and the familiar BPR positive-negative term can both involve $-\log \sigma(s^+ - s^-)$. But their derivations, pair construction, sampling, priors or regularizers, and use cases differ.

Do not infer that two complete methods are equivalent from one shared local loss expression. In particular, unobserved BPR items enter a relative-preference construction; they are not confirmed dislikes.

19.3 "Is listwise just an NDCG surrogate gradient?"

That description is closer to LambdaRank-style thinking than to every listwise objective.

For two items under the same query, the magnitude of the NDCG change caused by swapping their current positions can weight a pairwise gradient. Errors that affect the ranking metric more then receive more pressure.

A teaching expression is

$$\tilde{\ell}_{ij} = w_{ij} \log(1 + e^{-(s_i - s_j)}), \qquad w_{ij} = |\Delta \mathrm{NDCG}_{ij}|.$$

Usually the current ranking and weight are treated as fixed for that update rather than differentiated through a discrete sort. This explains a gradient construction; it does not turn exact discrete NDCG into an ordinary smooth function.

LambdaMART applies lambda ideas in boosted trees. Related gradient weighting can also train neural scorers. Neither observation means that every listwise method must work this way.

20. Listwise objectives are not all simulated NDCG gradients

This section is an **alignment supplement** to the spoken discussion. We did not independently derive every original listwise paper during the exchange. The minimal formulas prevent a mistaken identification of all listwise learning with NDCG-based lambdas.

20.1 ListNet-style top-one distributions

For n candidates under one query, define the prediction distribution

$$p_i = \frac{e^{s_i}}{\sum_j e^{s_j}}.$$

Transform relevance labels through a specified g to obtain a target distribution:

$$q_i = \frac{e^{g(y_i)}}{\sum_j e^{g(y_j)}}, \qquad L = -\sum_i q_i \log p_i.$$

The derivative is simple:

$$\frac{\partial L}{\partial s_i} = p_i - q_i.$$

Every document receives a scalar derivative, but the denominator of p depends on the whole candidate list. That is listwise coupling. No simulated NDCG gradient is needed.

20.2 Why it resembles in-batch CE

If the target distribution is one-hot on one positive, the expression becomes softmax CE over the candidate set.

The mathematics can match while the data semantics differ. In-batch retrieval negatives are sampled from some distribution; a ranking list may come from retrieval or logged exposure. Sampling probabilities, duplicates, logQ corrections, and eligibility cannot be ignored just because two equations look alike.

20.3 ListMLE models a permutation

For target permutation π, a classic objective is

$$L = -\sum_{t=1}^{n} \log \frac{e^{s_{\pi_t}}}{\sum_{j=t}^{n} e^{s_{\pi_j}}}.$$

The interpretation is sequential selection: choose the first item from the full set, then the second from those remaining, and continue. The product of those choices defines the target permutation likelihood.

Ties in relevance require a defined treatment. An arbitrary tie ordering should not silently become a certain preference.

This objective also has ordinary differentiable score terms; it does not need an NDCG surrogate first.

20.4 Consolidate the objectives

Objective	What determines the penalty	Source of score derivatives
Pointwise BCE	Individual score and label	$\sigma(s_i) - y_i$
Pairwise logistic	Preferred-minus-negative difference	Opposite-signed derivatives for the two scores
ListNet-style CE	Predicted and target list distributions	$p_i - q_i$
LambdaRank-style weighting	Pair order and metric impact	Metric-weighted pairwise lambdas

No family is universally best independently of the data and product objective.

21. Separate architecture, data, objective, and optimization in experiments

To attribute an improvement, avoid changing everything at once.

Experiment	Hold fixed	Change	Question answered
History encoder	Information cutoff, task, candidates, budget	Pooling / GRU / attention	Does sequence modeling add value?
Multi-interest capacity	Encoder family, candidates, loss	K=1 versus K>1, with capacity reported	Are multiple representations worth their cost?
Auxiliary supervision	Main architecture and task	Auxiliary objective and coefficient	Does intermediate supervision help?
Routing	Encoder, data, capacity	Hard / soft / balance term	How do specialization and stability change?
Ranking objective	Scorer, split, labels, sampling	BCE / pairwise / listwise	Does a new objective better match evaluation?

Report effect size, evaluation uncertainty, seed variability, training cost, inference latency, and cache/index requirements where material.

Preprocessing, history length, negative sampling, sampled versus full-candidate evaluation, model size, and training-step budgets can all change the result. An isolated AUC number does not identify the cause of improvement.

For a regularizer, inspect both its intended diagnostic and the main task. Better usage balance or more orthogonal vectors do not automatically mean better recommendations.

22. Interview preparation and a practical ML coding loop

My final assessment was that interview discussion requires a clear account of data, motivation, objective, and constraints. ML coding does not require independently reproducing every paper. The earlier two-tower mock can serve as a practical foundation.

22.1 Know the shapes

```
history_ids             [B, T]
history_embeddings      [B, T, d]
history_mask            [B, T]
user_state              [B, d]
multi_interest_states   [B, K, d]
candidate_embeddings    [B, N, d]
scores                  [B, N]
pair_scores             [B, 2]
```

Define B, T, K, N, and d before using them. Many implementation errors come from confusing an axis, not from misunderstanding a paper's motivation.

22.2 A minimal pairwise objective

```python
import torch
import torch.nn.functional as F

def pairwise_logistic_loss(pos_score, neg_score, weight=None):
    # pos_score / neg_score: [B]
    loss = F.softplus(neg_score - pos_score)
    if weight is None:
        return loss.mean()
    # Require finite, nonnegative weights and a positive weight sum.
    return (weight * loss).sum() / weight.sum()
```

This is a shape-level illustration, not a complete validated training project. This article did not execute it end to end. An implementation must specify empty batches, all-padding histories, zero total weights, and other boundaries appropriate to its input contract.

22.3 The checks that matter

- Are positive and negative paired under the correct query?
- Does the mask exclude padding rather than invent negatives?
- Has the label leaked into the prediction input?
- Do gradients reach embeddings, the history encoder, and the shared scorer?
- Are derivative signs sensible, and can a tiny dataset be overfit?
- Are evaluation mode and disabled autograd both handled correctly?
- Is the retrieval query candidate-independent, and is its index compatible?

"The core idea has converged" does not mean "every original detail and implementation is mastered." Align inputs, key computation, output, and loss; then validate a small implementation. That is more useful than indefinitely adding model names.

23. Correction log, unfinished details, and references

23.1 Corrections worth retaining

Initial shorthand	More accurate understanding
GRU combines the previous input with the current input	It combines the previous hidden state with current input
A gate is a ReLU-like threshold	A proportion gate normally uses sigmoid
States near plus/minus one are more important	Activation magnitude does not define semantic importance
Attention obviously represents semantics	The matching is explicit; its meaning is learned
An auxiliary gate loss guarantees the correct memory	A predictive task supplies pressure, not semantic guarantees
K interests means one K-dimensional output	It means K-by-d activations generated by an encoder
All non-winning interests should repel the positive	Non-winner does not mean negative
L2 normalization prevents unused branches	It controls scale, not participation
Soft labels solve collapse	This is soft routing, and it only changes gradient paths
f and P are the same usage rate	One counts hard choices; the other averages soft probabilities
Switch balancing is original MIND	It is an extension considered in our discussion
Massive data guarantees updates to every interest	Routing and optimization dynamics still matter
One score per document makes the task pointwise	The objective defines the supervision organization
All listwise learning simulates NDCG gradients	ListNet and ListMLE differentiate probability objectives

23.2 What this discussion did not fully derive

We did not reproduce every AUGRU detail, every MIND forward/backward implementation choice, theoretical properties of every LambdaLoss, or the full family of industrial routing algorithms.

For a deeper follow-up, select one architecture and one objective and run an interpretable small experiment with a fixed data protocol. The conceptual map is already sufficient to make that investigation focused.

23.3 References

1. Cho et al. **Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation.** 2014. Paper. This article consistently defines z as the candidate-write proportion.
2. Bahdanau et al. **Neural Machine Translation by Jointly Learning to Align and Translate.** ICLR 2015. Paper. Background on the fixed-vector bottleneck and attention.
3. Hidasi et al. **Session-based Recommendations with Recurrent Neural Networks.** Paper. GRU4Rec timeline.
4. Zhou et al. **Deep Interest Network for Click-Through Rate Prediction.** KDD 2018. Paper. Candidate-conditioned aggregation, covered in Part 1.
5. Zhou et al. **Deep Interest Evolution Network for Click-Through Rate Prediction.** AAAI 2019. Paper. Intermediate supervision and interest evolution.

6. Kang and McAuley. **Self-Attentive Sequential Recommendation.** ICDM 2018. Paper. Align the encoder and its original BCE objective separately.
7. Li et al. **Multi-Interest Network with Dynamic Routing for Recommendation at Tmall.** CIKM 2019. Paper. Routing, initialization, and label-aware selection.
8. Fedus et al. **Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity.** JMLR 2022. Paper. Top-1 balancing surrogate, not a component of original MIND.
9. Burges et al. **Learning to Rank using Gradient Descent.** ICML 2005. Official page. RankNet.
10. Rendle et al. **BPR: Bayesian Personalized Ranking from Implicit Feedback.** UAI 2009. Paper. The arXiv upload year differs from the original publication year.
11. Cao et al. **Learning to Rank: From Pairwise Approach to Listwise Approach.** ICML 2007. Official page. ListNet.
12. Xia et al. **Listwise Approach to Learning to Rank: Theory and Algorithm.** ICML 2008. DOI. ListMLE.
13. Burges. **From RankNet to LambdaRank to LambdaMART: An Overview.** 2010. Official page. Metric-weighted ranking gradients.
14. Yuan et al. **Tenrec: A Large-Scale Multipurpose Benchmark Dataset for Recommender Systems.** NeurIPS 2022. Project and tables.
15. Betello et al. **A Reproducible Analysis of Sequential Recommender Systems.** 2024. Paper. Configuration-dependent comparisons.

References ground the named methods. The objections, gradient expansions, experimental suggestions, and teaching variants are a reconstruction of this discussion, not a claim to reproduce each paper's entire contribution or original discovery sequence.